LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

Stack dan Queue

Oleh:

Amalia Soraya NIM. 2510817120002

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL
2026

LEMBAR PENGESAHAN

LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2 : Stack dan Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Amalia Soraya
NIM : 2510817120002

Menyetujui,	Mengetahui,
Asisten Praktikum	Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani	Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIM. 2310817310009	NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN i

DAFTAR ISI ii

DAFTAR GAMBAR iii

DAFTAR TABEL iv

SOAL 1 1

A. Source Code 1

B. Pembahasan 2

SOAL 2 4

A. Source Code 5

B. Output Program 7

C. Pembahasan 7

SOAL 3 9

A. Source Code 9

B. Pembahasan 11

SOAL 4 13

A. Source Code 14

B. Output Program 15

C. Pembahasan 15

TAUTAN GIT 17

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 2 7
Gambar 2. Screenshot Hasil Jawaban Soal 4 15

DAFTAR TABEL

Tabel 1. Source Code file stack.cpp 1
Tabel 2. Source Code file prob1.cpp 5
Tabel 3. Source Code file queue.cpp 9
Tabel 4. Source Code file prob2.cpp 14

SOAL 1

Pada file stack.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file stack.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 1. Source Code file stack.cpp

```
#include "stack.h"

void init(Stack* s) {
    s->top = s->data ;
}
bool isEmpty(const Stack* s) {
    return s->top == s->data;
}
bool isFull(const Stack* s) {
    return s->top == (s->data + (MAX - 1));
}
void push(Stack* s, int value) {
    if (!isFull(s)) {
        s->top++;
        *s->top = value;
    }
}
void pop(Stack* s) {
    if (!isEmpty(s)) {
        s->top--;
```

```
21      }
22  }
23  int peek(const Stack* s) {
24      if (!isEmpty(s)) {
25          return *(s->top);
26      }
27      return -1;
28  }
```

B. Pembahasan

Pada baris [1], `#include "stack.h"` digunakan untuk mengimpor header buatan. Header ini berisi deklarasi fungsi dan inisialisasi struct, yang nantinya akan dipakai pada program ini untuk implementasi fungsi dan pemakaian struct tersebut.

Pada baris [3] sampai [5], adalah fungsi inisialisasi. Fungsi ini berguna untuk menginisialisasi lokasi pointer top untuk menunjuk ke memori data. Cara kerjanya adalah pointer top yang awalnya tidak menunjuk kemanapun, jadi menunjuk ke s->data, yaitu variabel data untuk menyimpan stack. Variabel topnya menunjuk aja, tidak mengakses nilai dalam variabel data.

Pada baris [6] sampai [8], adalah fungsi untuk mengecek apakah stack kosong atau tidak. Fungsi ini digunakan untuk mengecek isi stack, apakah ada isinya atau tidak. Cara kerjanya dengan mengecek apakah yang ditunjuk s->top itu s->data atau bukan. Karna kalau s->top itu sama dengan s->data, berarti stack masih kosong. Fungsi ini mengembalikan nilai boolean (true atau false).

Pada baris [9] sampai [11] adalah fungsi cek stack full atau tidak. Fungsi ini mengembalikan nilai boolean, digunakan untuk mengecek apakah stack full atau tidak. Cara kerjanya dengan mengecek apakah pointer top menunjuk ke lokasi data maksimal atau bukan. `(s->data + (MAX - 1))` digunakan sebagai perbandingan, tempat paling atas dari stack. Kalau top menunjuk ke `(s->data + (MAX - 1))` berarti stack sudah penuh dan tidak bisa di-push lagi.

Pada baris [12] sampai [17] adalah fungsi push atau memasukkan nilai baru ke stack. Fungsi ini digunakan untuk menambah nilai baru ke dalam stack, di alamat memori yang

bersebelahan juga. Cara kerjanya adalah fungsi ini tidak return apapun, ia fungsi void. Pertama, fungsi ini memastikan kalau stacknya belum full dengan sintaks `if (!isFull(s))` . Kalau stacknya penuh, fungsi ini tidak berjalan. Kalau tidak penuh, pointer top geser petunjuk dulu ke memori sebelah atau memori atas dengan sintaks `s->top++;` , lalu mengakses variabel data dengan sintaks `*s->top = value;` dan memasukkan nilai terbaru ke alamat yang ditunjuk `s->top`.

Pada baris [18] sampai [22] adalah fungsi pop. Fungsi pop ini digunakan untuk menggeser pointer top, seakan akan nilai stack yang diatasnya dibuang atau dikeluarkan. Cara kerjanya adalah fungsi ini mengecek dulu apakah stacknya kosong atau tidak, kalau tidak, baru lah pointer top digeser ke memori sebelumnya. Fungsi ini tidak return apapun, dan hanya berjalan jika stack tidak kosong.

Pada baris [23] sampai [28] adlaah fungsi peek. Fungsi peek ini digunakan untuk melihat nilai terbaru dari stack, atau nilai teratas. Cara kerjanya adalah fungsi ini mengecek apakah stack kosong atau tidak, kalau tidak, pointer top mengakses/mengambil nilai yang ditunjuknya (di variabel data) untuk direturn dengan sintaks `*(s->top).` `*` atau deference ini simbol untuk variabel pointer, agar pointer mengambil nilai yang ditunjuknya gitu. Kalau stacknya kosong, fungsi ini return -1 (harus return karna ini fungsi yang mengembalikan interger).

Semua fungsi memiliki time complexity O(1) karena tidak ada fungsi yang bergantung pada nilai N.

SOAL 2

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru
- dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut,
- lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

- $1 \leqslant n \leqslant 100$
- $-1000 \leqslant Ai \leqslant 1000$

Keterangan:

- n adalah jumlah simbol dalam ekspresi
- Ai adalah nilai bilangan pada input
- Simbol yang valid hanya:
  - bilangan bulat
  - operator: +, -, *, /

Format Input

n

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Sebuah bilangan hasil dari kalkulator Si Palui.

Contoh Kasus

Input	Output
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

A. Source Code

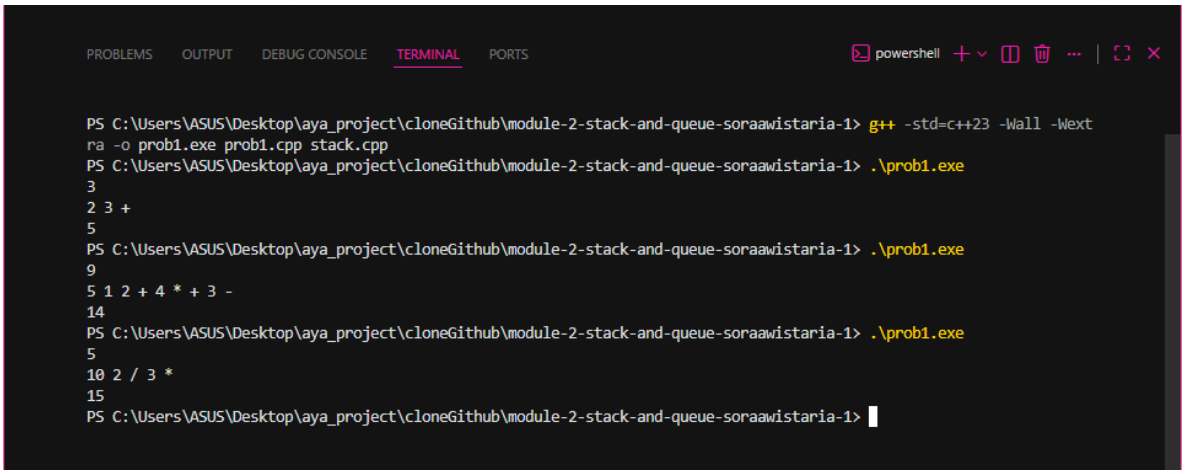
Tabel 2. Source Code file prob1.cpp

```
#include "stack.h"
#include <iostream>
using namespace std;
#include <string>

int main() {
    Stack s;
    init(&s);
    int N, angka, a, b, hasil;
    string x;

    cin >> N;
    for (int i = 0; i < N; i++) {
        cin >> x;

```

```
        if (x != "+" && x != "-" && x != "*" && x != "/") {
            angka = stoi(x);
            push(&s, angka);

        } else {
            if (isEmpty(&s)) {
                cout << "Stack masih kosong" << endl;
                return 1;
            }
            a = peek(&s);
            pop(&s);

            if (isEmpty(&s)) {
                cout << "Stack masih kosong" << endl;
                return 1;
            }
            b = peek(&s);
            pop(&s);

            if (x == "+") hasil = b + a;
            else if (x == "-") hasil = b - a;
            else if (x == "*") hasil = b * a;
            else if (x == "/") {
                if (a == 0){
                    cout << "Gabisa dibagi 0" << endl;
                }
                hasil = b / a;
            }
            push(&s, hasil);
        }
```

```
46        }
47        cout << peek(&s) << endl;
48        // cout << a << " " << b << " " << hasil << endl;
49        return 0;
50    }
```

B. Output Program



Gambar 1. Screenshot Hasil Jawaban Soal 2

C. Pembahasan

Pada baris [1] sampai [4] adalah header, import header. `#include "stack.h"` digunakan untuk mengimpor header buatan, yaitu stack.h yang ada di folder, lalu `#include <iostream>` digunakan untuk impor fungsi cin dan cout, lalu `using namespace std;` digunakan untuk menyingkat sintaks cin dan cout, dan `#include <string>` digunakan untuk impor fungsi string karena program ini akan menggunakan string.

Pada baris [7] sampai [10] adalah deklarasi dan inisialisasi variabel. `Stack s` digunakan untuk menggunakan struct stack yang diinisialisasi dengan s, `init(&s)` digunakan untuk inisialisasi pointer top pada data stack, `int N, angka, a, b, hasil` digunakan untuk deklarasi variabel interger yang akan dipakai pada program, `string x` digunakan untuk deklarasi variabel x sebagai string.

Pada baris [12] sampai [14] adalah pengambilan input. O digunakan untuk input nilai N (jumlah simbol ekspresi). O digunakan untuk mengambil nilai A_i dengan cara melakukan perulangan input sebanyak N. Nilai input A_i dimasukkan ke variabel x.

Masih di dalam perulangan, pada baris [16] sampai [18] adalah kode yang digunakan untuk mem-push nilai interger ke stack. Cara kerjanya adalah dengan memastikan bahwa input bukanlah operator bilangan. Jika input bukan merupakan operator bilangan, melainkan interger, maka string x tadi diubah menjadi nilai interger dan disimpan di variabel angka. Selanjutnya variabel angka akan mem-push nilai tersebut masuk ke dalam stack.

Pada baris [20] sampai [45] adalah kode yang berjalan jika inputnya operator bilangan. Kode baris [21] sampai [23] digunakan untuk mengecek apakah stack kosong atau tidak. Kalau kosong, print error dan program berhenti. Jika stack tidak kosong, kode berlanjut. Baris [25] dan [26] digunakan untuk melihat nilai stack yang terbaru, lalu disimpan pada variabel a, lalu pop atau geser pointer top. Lalu, kode [28] sampai [31] digunakan untuk mengecek apakah stack kosong atau tidak (sama seperti [21]-[23]). Jika tidak kosong, nilai stack terbaru akan disimpan di variabel b dan melakukan pop lagi.

Pada baris [35] sampai [42] adalah operasi bilangan. Kode ini digunakan untuk mengoperasikan 2 angka stack terluar atau nilai variabel a dan b tadi. Khusus pembagian, ada pesan error jika penyebutnya 0.

Pada baris [44] digunakan untuk push hasil operasi 2 nilai tadi ke stack, dan baris [47] digunakan untuk mengeprint atau menampilkan output berupa nilai hasil operasi tadi. Caranya adalah dengan menggunakan fungsi peek() yang mengintip nilai terbaru stack. Makanya hasil operasi tadi disimpan lagi dalam stack.

Time complexity nya adalah O(n) karena ada 1 for loop.

SOAL 3

Pada file queue.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file queue.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 3. Source Code file queue.cpp

```
#include "queue.h"

void init(Queue* q) {
    q->front = q->data;
    q->rear = q->data - 1;
}

bool isEmpty(const Queue* q) {
    return q->front == q->rear;
}

bool isFull(const Queue* q) {
    return q->rear == (q->data + (MAX -1));
}

void enqueue(Queue* q, int value) {
    if (!isFull(q)) {
        q->rear++;
        if(q->rear == q->data + MAX) {
            q->rear = q->data;
```

```
        }

        *q->rear = value;
    }
}

void dequeue(Queue* q) {
    if (!isEmpty(q)) {
        q->front++;

        if(q->front == q->data + MAX) {
            q->front = q->data;
        }
    }
}

int front(const Queue* q) {
    if(!isEmpty(q)){
        return *(q->front);
    }
    return -1;
}

int back(const Queue* q) {
    if(!isEmpty(q)){
        return *(q->rear);
    }
    return -1;
}
```

B. Pembahasan

Pada baris [1], `#include "queue.h"` digunakan untuk impor header buatan, yang isinya inisialisasi struct dan deklarasi fungsi.

Pada baris [3] sampai [6] adalah fungsi `init`. Fungsi ini digunakan untuk menginisialisasi pointer `front` dan pointer `rear`. Cara kerjanya adalah pointer `front` yang awalnya tidak menunjuk apapun, sekarang mengarah ke `q->data`, sehingga pointer `front` isinya alamat memori array data atau ya `front` menunjuk ke variabel `data`. Lalu pointer `rear` menyimpan memori sebelum array `data`, sehingga ia menunjuk -1. Kenapa menyimpan memori sebelum variabel `data`? Karena kalau ingin memasukkan nilai baru, harus geser pointer dulu ke memori berikutnya agar bisa menambahkan nilai.

Pada baris [8] sampai [10] adalah fungsi `isEmpty`. Fungsi ini digunakan untuk mengecek apakah queue kosong atau tidak. Cara kerjanya adalah fungsi ini mengecek apakah nomor memori `q->front` sama dengan `q->rear`. Kalau sama, berarti lokasi *head* dan *tail* sama, sehingga bisa dianggap queue masih kosong. Fungsi ini mereturn nilai true atau false.

Pada baris [12] sampai [14] adalah fungsi `isFull`. Fungsi ini digunakan untuk mengecek apakah queue kosong atau tidak. Cara kerjanya adalah dengan mengecek apakah pointer rear berisi memori tertinggi data atau tidak, atau dengan kata lain apakah pointer rear menyimpan memori/menunjuk ke puncak `q->data` atau tidak dengan sintaks `q->rear == (q->data + (MAX -1)).(q->data + (MAX -1))` adalah memori terakhir/batas memori array `data`. Fungsi ini mengembalikan nilai true atau false.

Pada baris [16] sampai [25] adalah fungsi `enqueue`. Fungsi enqueue berguna untuk memasukkan nilai ke dalam queue. Cara kerjanya adalah mengecek apakah queue sudah penuh atau belum, jika belum, geser pointer rear dan tambahkan nilai dengan sintaks `q->rear++` untuk menggeser pointer dan `*q->rear = value` untuk menambahkan nilai baru. Namun, queue bersifat First In First Out. Ketika dequeue, memori array pada indeks pertama akan kosong, sedangkan rear akan terus bertambah. Bisa jadi queue indeks awal kosong, namun dianggap penuh karna rear yang sudah menunjuk ke batas memori. Sehingga, queue dibuat circular. Dengan menambahkan syarat ”jika pointer rear sama dengan

menunjuk batas memori array, pointer rear dipindah ke memori awal array" bisa dilihat pada baris [19] dan [20].

Pada baris [27] sampai [35] adalah fungsi `dequeue`. Fungsi ini digunakan untuk menggeser pointer front seakan akan nilai pada memori sebelumnya dikeluarkan. Cara kerjanya adalah menggeser pointer front ke memori berikutnya jika dan hanya jika queue tidak kosong. Sama seperti enqueue, dequeue juga dibuat circular agar tidak melewatkan memori yang kosong, dengan sintaks seperti pada baris [31] dan [32].

Pada baris [37] sampai [42] adalah fungsi `front`. Fungsi ini digunakan untuk mengambil nilai yang ditunjuk pointer front pada variabel `data`, yaitu nilai depan queue. Fungsi ini mengembalikan nilai interger, sintaks untuk pointer agar bisa mengakses/mengambil nilai pada array adalah dengan menambahkan deference (`*`) seperti `*(q->front)`. Fungsi ini mengembalikan nilai yang ditunjuk pada variabel `data` jika dan hanya jika queue tidak kosong.

Pada baris [44] sampai [49] adalah fungsi `back`. Fungsi ini digunakan untuk mengambil nilai yang ditunjuk rear pada variabel data, yaitu nilai belakang queue. Fungsi ini mengembalikan nilai interger yang berasal dari variabel data jika dan hanya jika queue tidak kosong.

Semua fungsi memiliki time complexity O(1) karena tidak ada fungsi yang bergantung pada nilai N.

SOAL 4

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan.

Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leqslant n \leqslant 100$
- $-1 \leqslant k \leqslant n$
- $-1000 \leqslant Ai \leqslant 1000$

Keterangan:

- n: jumlah elemen
- k: ukuran jendela
- Ai: elemen array

Format Input

n k

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok yang dibentuk
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi.

Contoh Kasus

Input	Output
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

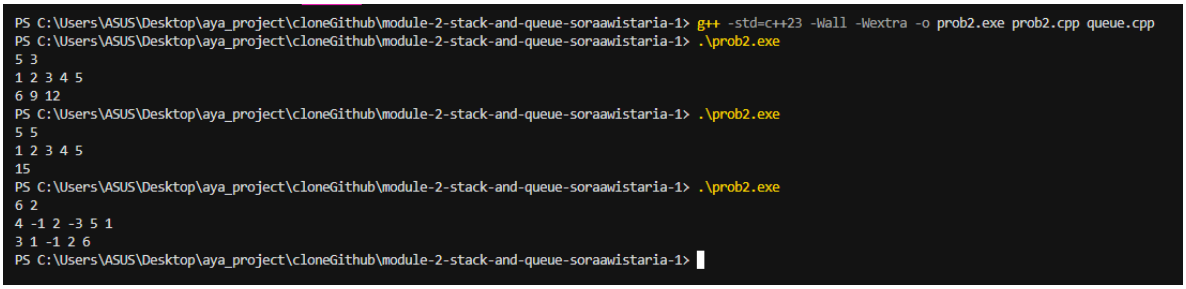
A. Source Code

Tabel 4. Source Code file prob2.cpp

```
#include "queue.h"
#include <iostream>
using namespace std;

int main() {
    Queue q;
    init(&q);
    int N, k;

    cin >> N >> k;
    int A[N];

    for (int i = 0; i < N; i++) {
        cin >> A[i];
    }

    int hasil = 0;
    for (int i = 0; i < k; i++){
        enqueue(&q, A[i]);
        hasil += A[i];
```

```
21      }
22      cout << hasil << " ";
23
24      for (int i = k; i < N; i++) {
25          int dihapus = front(&q);
26
27          dequeue(&q);
28          enqueue(&q, A[i]);
29
30          hasil = hasil - dihapus + A[i];
31
32          cout << hasil << " ";
33      }
34      return 0;
35
36
37  }
```

B. Output Program



```
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-2-stack-and-queue-soraawistaria-1> g++ -std=c++23 -Wall -Wextra -o prob2.exe prob2.cpp queue.cpp
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-2-stack-and-queue-soraawistaria-1> .\prob2.exe
5 3
1 2 3 4 5
6 9 12
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-2-stack-and-queue-soraawistaria-1> .\prob2.exe
5 5
1 2 3 4 5
15
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-2-stack-and-queue-soraawistaria-1> .\prob2.exe
6 2
4 -1 2 -3 5 1
3 1 -1 2 6
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-2-stack-and-queue-soraawistaria-1>
```

Gambar 2. Screenshot Hasil Jawaban Soal 4

C. Pembahasan

Pada baris [1] sampai [3] adalah header, import header. `#include "queue.h"` digunakan untuk mengimpor header buatan, yaitu queue.h yang ada di folder, lalu `#include <iostream>` digunakan untuk impor fungsi cin dan cout, dan `using namespace std;` digunakan untuk menyingkat sintaks cin dan cout.

Pada baris [6] sampai [8] adalah deklarasi dan inisialisasi variabel. `Queue q` digunakan untuk deklarasi struct queue yang diinisialisasi dengan q, `init(&q)` digunakan untuk inisialisasi pointer front dan rear pada data queue, `int N` dan `k` digunakan untuk deklarasi variabel interger yang akan dipakai pada program.

Pada baris [10] sampai [15] adalah proses input. `cin >> N >> k` digunakan untuk menginput nilai N sebagai jumlah elemen dan nilai k sebagai jumlah elemen yang ditambah. `int A[N]` digunakan untuk deklarasi variabel array `A` dengan ukuran `N`. `for (int i = 0; i < N; i++)` digunakan untuk loop, dengan isi `cin >> A[i]` digunakan untuk menginput nilai A_i satu persatu. Cara kerjanya adalah input user disimpan di variabel `N` dan k, lalu nilai variabel `N` ini digunakan untuk ukuran array `A` dan batas loop untuk input A_i.

Baris [17] adalah inisialisasi variabel hasil, dinilai 0 agar bisa ditambah dengan hasil pertambahan nanti. Baris [18] sampai [21] digunakan untuk perulangan pertambahan nilai per `k` tadi. Cara kerjanya, pertama tuh bikin loop dulu dari 0 sampai `k`, karna `k` ini jumlah elemen yang akan ditambah-tambah. Lalu, nilai yang di variabel `A[i]` akan di enqueue atau dimasukkan ke queue. Lalu nilai `A[i]` dipakai lagi untuk ditambah dengan nilai variabel hasil. Jadi, misal nilai k nya 2, maka perulangannya 2 kali, dan variabel hasil itu ditambah per 2 nilai yang ada di array `A[i]`. `cout << hasil << " ";` digunakan untuk menampilkan output nilai variabel hasil tadi setelah ditambah-tambah.

Pada baris [24] sampai [33] adalah inti dari logika sliding window. Cara kerjanya adalah melakukan perulangan dari index `k` hingga `N`. Di setiap perulangan, `front(&q)` digunakan untuk melihat angka yang akan keluar dari jendela, lalu `dequeue(&q)` mennggeser pointer seakan membuang angka tersebut dari queue. Selanjutnya, angka baru `A[i]` di-enqueue ke dalam queue. Untuk memperbarui jumlah total tanpa harus menjumlahkan ulang semua elemen, gunakan rumus `hasil = hasil - dihapus + A[i]`. Ini sangat efisien karena hanya mengurangi angka yang keluar dan menambah angka yang baru masuk. Hasil setiap kelompok kemudian dicetak ke layar dengan spasi sebagai pemisah.

Time complexitynya adalah O(n) karena for loop nya bergantian.

TAUTAN GIT

Repository : https://github.com/DSA26-ULM/module-2-stack-and-queue-soraawistaria-1